Branch: MCA (Data Science) Kargil	**Semester:** 2
Student Name: Sameer Shahi	**UID:** 25MCD10025
Subject Name: Technical Training - I Lab	**Subject Code**: 25CAP-652
Section/Group: 25MCD KAR-1	**Date of Performance:** 31-03-2026

Experiment 8

Experiment Aim:

To apply the concept of Stored Procedures in database operations in order to perform tasks like insertion, updating, deletion, and retrieval of data efficiently, securely, and in a reusable manner within the database system.

Tools used: PostgreSQL

Objectives:

- **Procedure Implementation:** To understand how to create and use stored procedures for database operations.
- **Code Reusability:** To reduce redundancy by encapsulating logic inside procedures.
- **Data Manipulation:** To perform INSERT, UPDATE, DELETE operations using procedures.

- **Data Retrieval:** To understand the use of functions for returning result sets.

Theory:

Stored Procedures are precompiled SQL code blocks stored in the database that can be executed repeatedly. They improve performance, enhance security, and promote modular programming. PostgreSQL supports procedures (for actions) and functions (for returning data).

Experiment Steps:

Step 1: Creating Table

Query:

```
CREATE TABLE Employee (
emp_id INT PRIMARY KEY,
name VARCHAR(50),
salary INT
);
```

Step 2: Inserting Sample Data **Query:**

```
INSERT INTO Employee VALUES
(1, 'Rahul', 30000),
(2, 'Priya', 35000),
(3, 'Amit', 40000),
```

(4, 'Neha', 45000),

(5, 'Karan', 38000),

(6, 'Simran', 42000),

(7, 'Arjun', 50000),

(8, 'Meera', 36000);

Step 3: Stored Procedure for INSERT **Query:**

```
CREATE OR REPLACE PROCEDURE add_employee(
p_id INT,
p_name VARCHAR,
p_salary INT )
LANGUAGE plpgsql
AS $$
BEGIN
INSERT INTO Employee(emp_id, name, salary)
VALUES (p_id, p_name, p_salary);
END;
$$;
```

Step 4: Stored Procedure for UPDATE

Query:

```
CREATE OR REPLACE PROCEDURE update_salary( p_id INT, p_salary INT )
LANGUAGE plpgsql
AS $$
```

```
BEGIN
UPDATE Employee
SET salary = p_salary
WHERE emp_id = p_id;
END;
$$;
```

Step 5: Stored Procedure for DELETE

Query:

```
CREATE OR REPLACE PROCEDURE delete_employee( p_id
INT
)
LANGUAGE plpgsql
AS $$
BEGIN
DELETE FROM Employee
WHERE emp_id = p_id;
END;
$$;
```

Step 6: Function for SELECT (Retrieval)

Query:

```
CREATE OR REPLACE FUNCTION get_all_employees()
RETURNS TABLE(emp_id INT, name VARCHAR, salary INT)
```

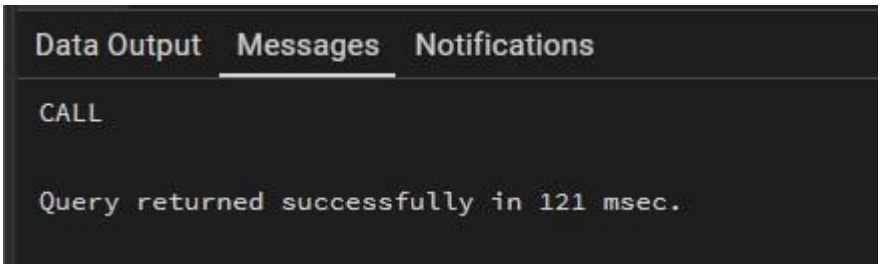
```
LANGUAGE plpgsql
AS $$
BEGIN
RETURN QUERY
SELECT * FROM Employee;
END;
$$;
```

Step 7: Calling Procedures and Function

Query:

```
-- Insert
CALL add_employee(9, 'Rohan', 37000);
```

Output:




```
-- Update
CALL update_salary(2, 40000);
```

Output:

```
Data Output   Messages   Notifications
CALL

Query returned successfully in 121 msec.
```

-- Delete

CALL delete_employee(3);

Output:

```
Data Output   Messages   Notifications
CALL

Query returned successfully in 121 msec.
```

-- Retrieve

SELECT * FROM get_all_employees();

Output:

Data Output Messages Notifications

Showing rows: 1 to 8 Page No: 1 of 1

	emp_id integer	name character varying	salary integer
1	1	Rahul	30000
2	4	Neha	45000
3	5	Karan	38000
4	6	Simran	42000
5	7	Arjun	50000
6	8	Meera	36000
7	9	Rohan	37000
8	2	Priya	40000

Outcomes:

- Students will be able to create and execute stored procedures.
- Students will understand the difference between procedures and functions.
- Students will perform CRUD operations using procedures.
- Students will retrieve data using functions in PostgreSQL.